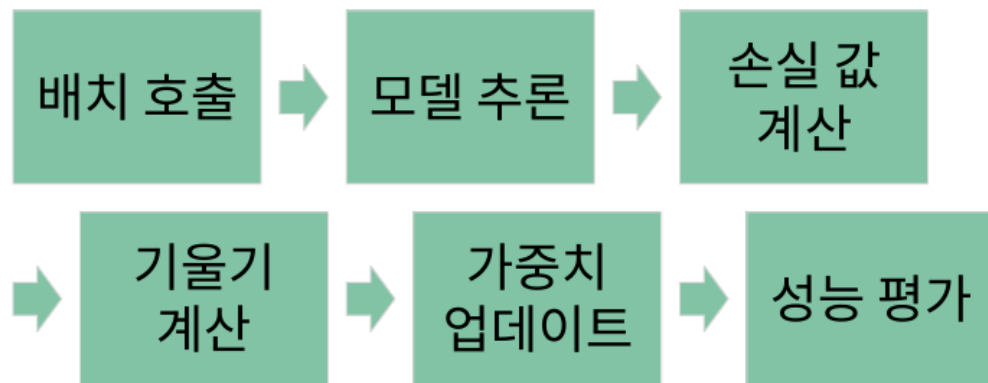


26.06.30 (1)

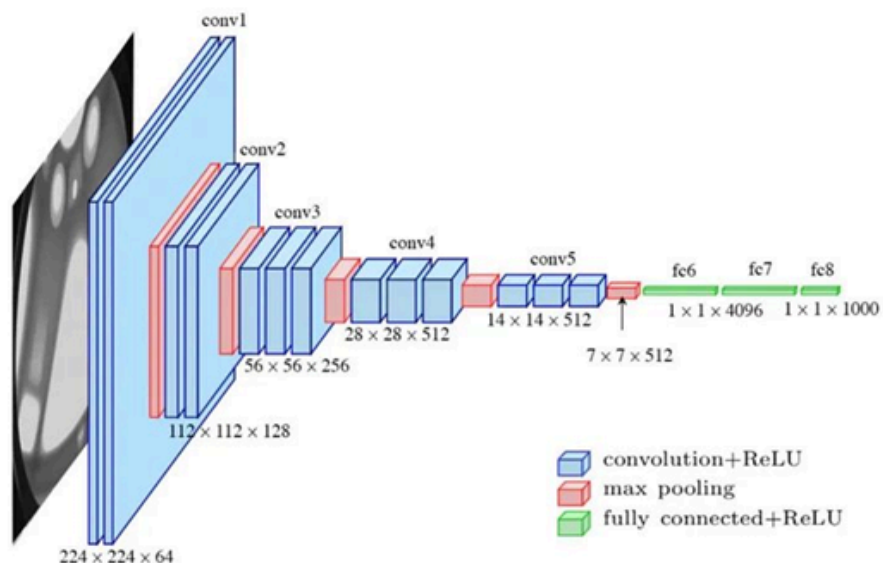
손실함수와 평가지표

모델 학습 프로세스



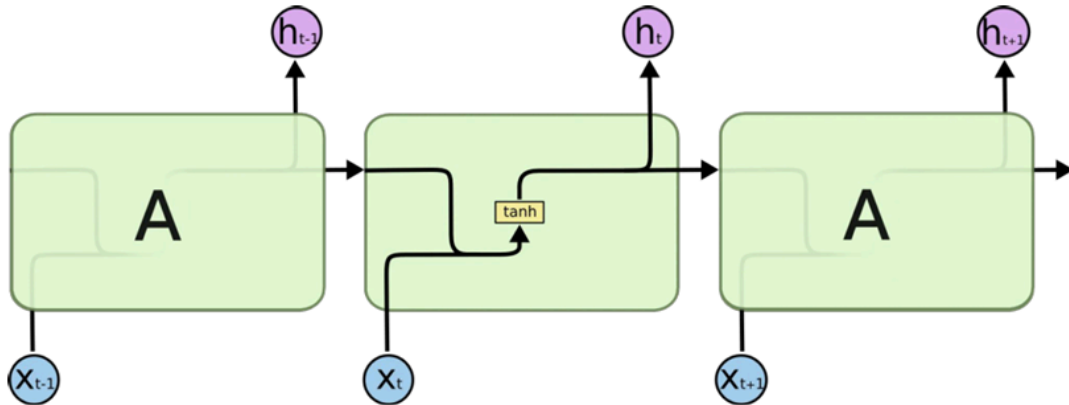
- 학습루프 : 배치 호출 → 모델 추론 → 손실 값 계산 → 기울기 계산 → 가중치 업데이트

CNN 모델의 추론



- $7 \times 7 \times 512 \rightarrow 1 \times 1 \times 4096 \rightarrow 1 \times 1 \times 1000$ (클래스마다 각각의 점수를 구한다)

RNN 모델의 추론



출력층이 MLP를 사용하는 이유

- Representation vs Decision
- 대부분의 task는 결국 벡터 $\rightarrow$ 값의 문제
- 표현공간에서 이미 선형적으로 분리되도록 학습
- 일반화 성능과 안정성
- 확률 해석과 loss 함수

이유	개념	영화 리뷰 데이터 예시
Representation vs Decision	RNN은 문장을 이해해서 벡터(표현) 를 만들고, MLP는 그 벡터를 보고 최종 판단 만 한다.	RNN $\rightarrow$ 리뷰 이해 / MLP $\rightarrow$ 긍정·부정 결정
벡터 $\rightarrow$ 값 변환	대부분의 AI 문제는 마지막에 벡터를 하나의 값 (또는 확률)으로 바꾸는 작업이다.	128차원 벡터 $\rightarrow$ 긍정 확률 0.91
선형 분리 가능	RNN이 이미 긍정과 부정을 구분하기 쉬운 벡터를 만들기 때문에 마지막은 간단한 Linear만으로도 충분하다.	😊 긍정 벡터 / 😞 부정 벡터를 선 하나로 구분
일반화 성능	출력층을 너무 복잡하게 만들면 학습 데이터만 외우는 **과적합(Overfitting)** 이 생길 수 있다.	Linear 하나가 더 안정적
Loss 함수와 잘 연결 됨	Linear 출력 $\rightarrow$ Sigmoid $\rightarrow$ BCE Loss처럼 확률 계산이 쉽다.	영화 리뷰 긍정 확률 계산

출력층이 MLP가 아닌 경우

- Metric Learning (triplet)
- Diffusion (noise prediction)
- RL (policy + value)
- Multi-modal alignment(multi-projection)

분야	출력하는 것	왜 MLP를 안 쓰는가?
Metric Learning	벡터(Embedding)	분류가 아니라 비슷한 데이터끼리 가깝게 만드는 것이 목적
Diffusion	Noise(노이즈)	이미지를 생성하기 위해 노이즈를 예측 해야 함
Reinforcement Learning (RL)	행동(Action), 가치(Value)	정답 클래스를 맞추는 것이 아니라 어떤 행동을 할지 결정
Multi-modal Alignment (CLIP 등)	이미지 벡터 + 텍스트 벡터	이미지와 문장을 같은 벡터 공간으로 맞추는 것 이 목적

손실함수

모델이 예측한 값과 실제 값의 차이를 수치로 표현한 함수

- Regression : MSE
- Classification : Cross-Entropy

우리가 정말 풀고 싶은 문제

Classification → Accuracy 최대화

Machine Translation → 정확한 문장 생성

Retrieval → Ranking 품질

이런 지표는 대부분 미분 불가, Surrogate loss 사용

Surrogate Loss의 한계

- 학습에는 유리하지만 목표와 괴리
- classification : accuracy는 decision boundary 만 중요하지만 cross - entropy는 확률 분포를 맞추려고함
- next token prediction : token 단위로는 잘 맞추지만 전체 문장이 어색할 수 있음

Metric

- loss는 학습용, metric은 평가용
- loss의 핵심 요건 : 미분 가능
- metric의 핵심 요건 : 실제 성능을 반영

항목	개념	예시
Loss	모델을 학습 시키기 위한 함수	BCE Loss, CrossEntropy, MSE
Metric	모델 성능을 평가 하기 위한 지표	Accuracy, F1, MAE

구분	Loss	Metric
목적	학습	평가
미분 가능?	✅ 반드시 가능	❌ 필요 없음
Optimizer 사용	O	X
예시	BCE, CrossEntropy, MSE	Accuracy, F1, BLEU

Classification Metric (분류)

- Accuracy
- precision/Recall/F1
- AUROC

Metric	개념	예시
Accuracy	전체 중 맞춘 비율	영화 리뷰 긍정/부정
Precision	Positive라고 예측한 것 중 진짜 Positive 비율	스팸 메일 탐지
Recall	실제 Positive 중 찾아낸 비율	암 진단
F1 Score	Precision과 Recall의 균형	데이터 불균형 문제
AUROC	Threshold 변화에도 얼마나 잘 분류하는지	의료 AI

Regression Metric (회귀)

- MAE / MSE / RMSE
- R2

Metric	개념	예시
MAE	평균 절대 오차	집값 예측
MSE	평균 제곱 오차	주가 예측

Metric	개념	예시
RMSE	MSE의 제곱근	자동차 속도 예측
R^2	모델 설명력	회귀 모델 성능 비교

Ranking / Retrieval (검색)

- MRR
- NDCG
- Recall@K

Metric	개념	예시
MRR	정답이 몇 번째에 있는지	검색엔진
NDCG	검색 순위 품질	추천 시스템
Recall@K	상위 K개 안에 정답이 있는지	RAG, 문서 검색

이미지 task Metric

- IoU
- mAP

Metric	개념	예시
IoU	예측 영역과 실제 영역이 얼마나 겹치는지	자율주행 객체 탐지
mAP	객체 탐지 전체 성능	YOLO, Faster R-CNN

NLG Metric (자연어 생성)

- 답이 여러개일 수 있음
- exact match 보다는 의미 유사성이 중요
- BLEU, ROUGE 등

Metric	개념	예시
BLEU	번역 품질	영어 ↔ 한국어 번역
ROUGE	요약 품질	뉴스 요약

LLM에서의 Metric

- Human Evaluation
- LLM-as-a-judge
- Preference Modeling
- Embedding-based Metrics (bert score)

Metric	개념	예시
Human Evaluation	사람이 직접 평가	GPT 답변 품질
LLM-as-a-Judge	다른 LLM이 평가	GPT vs Claude 비교
Preference Modeling	사람이 여러 답변을 보고 더 좋은 답을 선택하여 AI가 그 선호를 학습	ChatGPT 학습 (RLHF)
Embedding-based Metrics	의미 유사도 비교	문장 유사도 평가

Benchmark

- 문제 정의
- 표준화된 평가 기준
- 모델 간 비교
- 간접적인 일반 능력 측정

역할	개념	예시
문제 정의	어떤 능력을 평가할지 결정	수학, 코딩, 추론
표준화된 평가	모두 같은 데이터셋 사용	MMLU
모델 비교	모델 간 성능 비교	GPT vs Claude vs Gemini
일반 능력 측정	종합적인 AI 능력 평가	HumanEval, GSM8K

요약

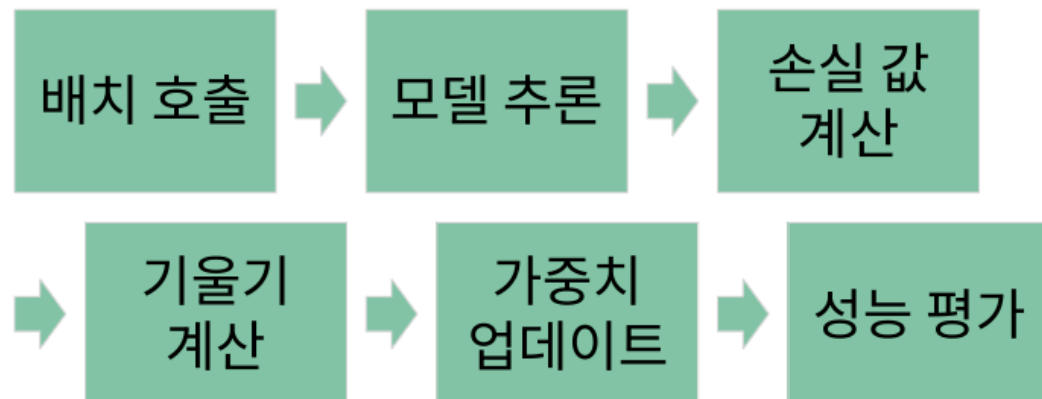
Task	개념	대표 Metric	개념
Classification (분류)	여러 클래스 중 하나를 예측	Accuracy, Precision, Recall, F1, AUROC	맞게 분류했는지 평가
Regression (회귀)	연속적인 숫자 예측	MAE, MSE, RMSE, R ²	숫자를 얼마나 정확하게 예측했는지

Task	개념	대표 Metric	개념
Ranking / Retrieval (검색·순위)	관련 문서를 얼마나 잘 찾는지	MRR, NDCG, Recall@K	정답 문서를 상위에 찾는 능력
Image Task (컴퓨터 비전)	이미지 분류, 객체 탐지, 분할	IoU, mAP	물체를 얼마나 정확하게 찾았는지
NLG (Natural Language Generation)	문장 생성, 번역, 요약	BLEU, ROUGE	생성한 문장이 얼마나 자연스럽게 정답과 비슷한지
LLM Evaluation	GPT 같은 거대언어 모델 평가	Human Evaluation, LLM-as-a-Judge, Preference Modeling, Embedding-based Metrics	사람이 봤을 때 좋은 답인지 평가

Gradient Descent (경사 하강법)

- Loss가 감소하는 방향으로 파라미터(가중치)를 조금씩 업데이트하는 최적화 알고리즘

모델 학습 프로세스



- 기울기 계산 → 가중치 업데이트 부분

모델 학습이란?

loss function을 최소화하는 파라미터 θ 를 찾는 문제

$$\theta^* = \arg \min_{\theta} L(\theta)$$

기호	의미	개념
θ (Theta)	모델의 모든 파라미터, 모델 안에서 학습되는 모든 값	학습해야 하는 가중치(Weight)와 편향(Bias)
$L(\theta)$	Loss Function	현재 모델이 얼마나 틀렸는지 나타내는 값
$\min$	Minimum	Loss를 최대한 작게 만든다.
$\argmin$	Argument of Minimum	Loss가 가장 작아지는 θ (파라미터)를 찾는다.
θ^*	최적의 파라미터	학습이 완료된 모델의 최종 가중치

Gradient Descent

Loss가 감소하는 방향으로 조금씩 이동

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$$

기호	의미	개념
θ_t	현재 파라미터 (Weight)	현재 모델이 가지고 있는 가중치

기호	의미	개념
θ_{t+1}	업데이트된 파라미터	다음 학습 후의 새로운 가중치
$L(\theta)$	Loss Function	현재 모델의 오차
$\nabla \theta L(\theta)$	Gradient(기울기)	Loss가 가장 빠르게 증가하는 방향
η (Eta)	Learning Rate	한 번에 얼마나 이동할지 결정하는 값, 가중치를 한번에 얼마나 수정할 것인지

- 새로운 가중치 = 현재 가중치 - (학습률 × Loss의 기울기)

GD 대신 SGD를 사용하는 이유

- 계산 효율
- 메모리 절약
- local minimum /saddle point에 강건

항목	Gradient Descent (GD)	Stochastic Gradient Descent (SGD)
학습 데이터 사용	전체 데이터 사용	일부 데이터(Batch 또는 1개) 사용
속도	느림	빠름
메모리 사용	많음	적음
Weight 업데이트	Epoch당 1번	Batch마다 여러 번
실제 사용	거의 안 함	대부분의 딥러닝 모델

GD 대신 SGD를 사용하는 이유	설명	쉽게 이해하기
계산 효율	전체 데이터를 모두 계산하지 않고 Batch 단위로 계산하므로 훨씬 빠르다.	10만 개를 한 번에 계산하는 대신 32개씩 계산
메모리 절약	한 번에 모든 데이터를 GPU/메모리에 올릴 필요가 없다.	큰 데이터셋도 학습 가능
Local Minimum / Saddle Point에 강함	Batch마다 Gradient가 조금씩 달라져 한 곳에 갇힐 가능성이 줄어든다	